

VD: tạo các class như interface

```
public interface Shape {
    void draw();
}
```

```
public interface Color {
    String get color();
}
```

```
public class Rectangle implements Shape, Color {
```

```
    @Override
```

```
    public void draw() {
```

```
        System.out.println("Draw" + this.get color() + "rectangle");
    }
```

```
    @Override
```

```
    public String get color() {
```

```
        return "Red";
    }
```

```
}
```

BT 1

```
public interface Mail {
```

```
    void method mail(); sendMail();
```

```
    void log;
```

```
}
```

```
class MailImpl implements Mail {
```

```
    @Override
```

```
    public void sendMail() {
```

```
        System.out.println("Đang gửi mail");
    }
```

```
    @Override
```

```
    public void log() {
```

```
        System.out.println("Đang ghi log");
    }
```

```
}
```

```
class Main {
```

```
    public static void main (String[] args) {
```

```
        Mail myMail = new MailImpl();
```

```
        myMail.sendMail();
```

```
        myMail.log();
    }
```

```
}
```


BT 2

```
public interface Shape {
    double getArea();
    double getPerimeter();
    void draw();
}
```

```
public class Rectangle implements Shape {
    private double width;
    private double height;
```

```
    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }
```

```
    @Override
    public double getArea() {
        return width * height;
    }
```

```
    @Override
    public double getPerimeter() {
        return 2 * (width + height);
    }
```

```
    @Override
    public void draw() {
        System.out.println("Drawing a");
    }
    public class Circle implements Shape {
        private double radius;
        public Circle(double radius) {
            this.radius = radius;
        }
```

```
    @Override
    public double getArea() {
        return Math.PI * radius * radius;
    }
```

```
    @Override
    public double getPerimeter() {
        return 2 * Math.PI * radius;
    }
```

```
    @Override
    public void draw() {
        System.out.println("Vẽ hình tròn");
    }
}
```

```
public class Main
```

```
{
    public static void main(String[] args)
```

```
{
    Shape s1 = new Rectangle(5, 4);
    Shape s2 = new Circle(3);
```

```
    s1.draw();
```

```
    s2.draw();
}
```


Bt 3

```
public class HangSanXuat {  
    private String tenHangSanXuat;  
    private String tenQuocGia;  
  
    public HangSanXuat (String tenHangSanXuat, String tenQuocGia) {  
        this.tenHangSanXuat = tenHangSanXuat;  
        this.tenQuocGia = tenQuocGia;  
    }  
}
```

6

public abstract class

PhuongTienDiChuyen

protected String loaiPhuongTien;
protected String sanXuat;

```
public PhuongTienDiChuyen (String loaiPhuongTien, String sanXuat) {  
    this.loaiPhuongTien = loaiPhuongTien;  
    this.hangSanXuat = hangSanXuat;  
}
```

```
public String layTenHangSanXuat () {  
    return hangSanXuat.getHangSanXuat();  
}
```

```
public void batDau () {  
    System.out.println ("Bat dau...");  
}
```

```
public void tangToc () {  
    System.out.println ("Tang toc...");  
}
```

```
public void duyLai () {  
    System.out.println ("Duy lai...");  
}
```

```
public abstract double layVayVanta ();  
}
```

public class MayBay extends PhuongTienDiChuyen {

private String loaiNhienLieu;

public MayBay (HangSanXuat hangSanXuat, String loaiNhienLieu) {

super ("May Bay", hangSanXuat);

this.loaiNhienLieu = loaiNhienLieu;

```
public void catCanh () {  
    System.out.println ("Cat canh");  
}
```

```
public void haCanh () {  
    ("Ha canh");  
}
```

public class XeOto extends PhuongTienDiChuyen {

private String loaiNhienLieu;

public XeOto (

super ("Xe oto", hangSanXuat);

this.loaiNhienLieu = loaiNhienLieu;

public class XeMay extends PhuongTienDiChuyen {

public XeMay (HangSanXuat hangSanXuat) {

super ("Xe May", hangSanXuat);

}

BT4

Public class B...

```

public abstract class Car {
    protected String color;
    protected double speed;
    public Car (String color, double speed) {
        this.color = color;
        this.speed = speed;
    }

```

```

    abstract void accelerate ();
    public String get color () {
        return color;
    }

```

```

    public double get speed () {
        return speed;
    }
}

```

```

public class Bmw extends Car {
    public Bmw (String color, double speed) {
        super (color, speed);
    }
}

```

```

    Civic extends Car {
        Civic (String color, double speed) {
            super (color, speed);
        }
    }
}

```

BT5

```

public interface Moveable {
    void moveUp ();
    void moveDown ();
    void moveLeft ();
    void moveRight ();
}

```

```

class MovePoint implements Moveable {
    int x, y, xSpeed, ySpeed;
}

```

```

    public Moveable point (int x, int y, int xSpeed, int ySpeed) {
        this.x = x;
        this.y = y;
        this.xSpeed = xSpeed;
        this.ySpeed = ySpeed;
    }
}

```